R2-A5-S11 INFORME – REDES GAN (DEEP LEARNING)

Carlos Eduardo Riaño Rodriguez

Bryan Felipe Roa Alvarado

Universidad de Cundinamarca

Inteligencia Artificial

Cundinamarca

Inteligencia Artificial Conceptos

Agosto 2026

Tabla de contenido

1. Resumen 3

2. Conceptos Clave 4

3. Descripción de la Práctica 5

3.1 Entorno y herramientas 5

3.2 Conjunto de datos 5

3.3 Arquitectura de los modelos 6

3.4 Configuración del entrenamiento 7

4. Resultados Obtenidos 8

4.1 Evolución de las pérdidas durante el entrenamiento 8

4.2 Evolución visual de las imágenes generadas 9

5. Análisis de Resultados 11

5.1 Comportamiento adversarial de las pérdidas 11

5.2 Calidad de las imágenes generadas 11

5.3 Relación con los conceptos teóricos 12

6. Conclusiones 12

7. Referencias 14

1. Resumen

El presente informe describe la implementación y evaluación de una Red Generativa Antagónica (GAN, por sus siglas en inglés Generative Adversarial Network) convolucional profunda, entrenada sobre el conjunto de datos MNIST para la generación de imágenes sintéticas de dígitos manuscritos. La práctica se desarrolló en el entorno Google Colab con TensorFlow 2.21.0 y aceleración por GPU NVIDIA T4. Se construyeron dos modelos: un generador basado en capas densas y convoluciones transpuestas (2 330 944 parámetros) y un discriminador convolucional (212 865 parámetros), entrenados de forma adversarial durante 10 épocas con el optimizador Adam (tasa de aprendizaje 1×10^{-4}) y una función de pérdida de entropía cruzada binaria. Los resultados muestran que, tras 10 épocas, el generador produjo imágenes que reproducen la estructura general de los dígitos, aunque con presencia de ruido y trazos difusos, comportamiento esperado para un entrenamiento corto. Las pérdidas del generador y del discriminador oscilaron de manera complementaria (G: 0.77 → 1.04; D: 1.03 → 1.15), reflejando la dinámica del juego minimax. Se concluye que la GAN aprendió la distribución subyacente de los datos, evidenciando el potencial del aprendizaje adversarial como técnica de generación de contenido sintético.

Palabras clave: *GAN, aprendizaje adversarial, generador, discriminador, deep learning, MNIST.*

2. Conceptos Clave

Redes Generativas Antagónicas (GAN)

Las Redes Generativas Antagónicas, propuestas por Goodfellow et al. (2014), constituyen una clase de modelos de aprendizaje profundo en los que dos redes neuronales —un generador y un discriminador— se entrenan simultáneamente mediante un juego de suma cero. El objetivo global es que el generador aprenda la distribución de probabilidad de un conjunto de datos reales y sea capaz de producir muestras nuevas y verosímiles a partir de ruido aleatorio.

Estructura: Generador y Discriminador

El Generador (G) recibe un vector de ruido latente z (en esta práctica, de dimensión 100) y lo transforma, mediante capas densas y convoluciones transpuestas, en una muestra sintética con la misma forma que los datos reales (imágenes de 28 × 28 píxeles). Su objetivo es engañar al discriminador produciendo muestras indistinguibles de las reales.

El Discriminador (D) recibe una imagen (real o generada) y emite una puntuación que indica la probabilidad de que sea real. Su objetivo es detectar correctamente las muestras falsas.

Aprendizaje Adversarial

El entrenamiento se formula como un juego minimax en el que el generador minimiza y el discriminador maximiza la siguiente función de valor:

$$min_G\ max_D\ V(D, G) = E[log\ D(x)] + E[log(1 - D(G(z)))]$$

En la práctica, el generador y el discriminador se actualizan alternativamente en cada paso. Conforme avanza el entrenamiento, el generador produce imágenes más realistas y el discriminador incrementa su capacidad de detección. El equilibrio ideal (equilibrio de Nash) se alcanza cuando el generador reproduce perfectamente la distribución real y el discriminador ya no puede distinguir entre ambas (precisión del 50 %).

Aplicaciones de las GAN

Las GAN se emplean en la actualidad en tareas como: síntesis de imágenes fotorrealistas (p. ej., StyleGAN), aumento de datos para entrenar otros modelos, transferencia de estilo artístico, superresolución de imágenes, generación de rostros, síntesis de voz, traducción imagen-a-imagen (CycleGAN) y generación de datos tabulares sintéticos para preservar privacidad.

3. Descripción de la Práctica

3.1 Entorno y herramientas

La práctica se desarrolló en Google Colab, con las siguientes características técnicas: framework TensorFlow 2.21.0 con API Keras; hardware GPU NVIDIA T4 (verificada mediante tf.config.list_physical_devices('GPU')); lenguaje Python 3.13; y librerías auxiliares NumPy y Matplotlib.

3.2 Conjunto de datos

Se utilizó el dataset MNIST (Modified National Institute of Standards and Technology), compuesto por 60 000 imágenes de entrenamiento de dígitos manuscritos (0–9) en escala de grises, con resolución de 28 × 28 píxeles. Las imágenes se normalizaron del rango original [0, 255] al intervalo [−1, 1] y se les añadió una dimensión de canal para obtener tensores de forma (28, 28, 1). Los datos se agruparon en lotes de 256 imágenes (BATCH_SIZE = 256), resultando en 234 lotes por época.

3.3 Arquitectura de los modelos

Tabla 1

Arquitectura del Generador

Capa	Tipo	Forma de salida	Parámetros
1	Dense	(None, 12544)	1 254 400
2	BatchNormalization	(None, 12544)	50 176
3	LeakyReLU	(None, 12544)	0
4	Reshape	(None, 7, 7, 256)	0
5	Conv2DTranspose (128, 5×5, s=1)	(None, 7, 7, 128)	819 200
6	BatchNormalization	(None, 7, 7, 128)	512
7	LeakyReLU	(None, 7, 7, 128)	0
8	Conv2DTranspose (64, 5×5, s=2)	(None, 14, 14, 64)	204 800
9	BatchNormalization	(None, 14, 14, 64)	256
10	LeakyReLU	(None, 14, 14, 64)	0
11	Conv2DTranspose (1, 5×5, s=2, tanh)	(None, 28, 28, 1)	1 600
	Total		2 330 944

Tabla 2

Arquitectura del Discriminador

Capa	Tipo	Forma de salida	Parámetros
1	Conv2D (64, 5×5, s=2)	(None, 14, 14, 64)	1 664
2	LeakyReLU	(None, 14, 14, 64)	0
3	Dropout (0.3)	(None, 14, 14, 64)	0
4	Conv2D (128, 5×5, s=2)	(None, 7, 7, 128)	204 928
5	LeakyReLU	(None, 7, 7, 128)	0
6	Dropout (0.3)	(None, 7, 7, 128)	0
7	Flatten	(None, 6272)	0
8	Dense (1, sin activación)	(None, 1)	6 273
	Total		212 865

3.4 Configuración del entrenamiento

La función de pérdida utilizada fue la entropía cruzada binaria (BinaryCrossentropy(from_logits=True)). El discriminador minimiza la suma de la pérdida sobre muestras reales (etiqueta 1) y muestras falsas (etiqueta 0); el generador minimiza la pérdida sobre sus muestras falsas etiquetadas como reales. Se empleó el optimizador Adam con tasa de aprendizaje 1×10^{-4} para ambas redes, durante 10 épocas, con una dimensión de ruido latente de 100 y 16 imágenes fijas de visualización (semilla constante).

4. Resultados Obtenidos

4.1 Evolución de las pérdidas durante el entrenamiento

La Tabla 3 presenta los valores de pérdida del generador (G) y del discriminador (D) al cierre de cada una de las 10 épocas, junto con el tiempo de cómputo por época.

Tabla 3

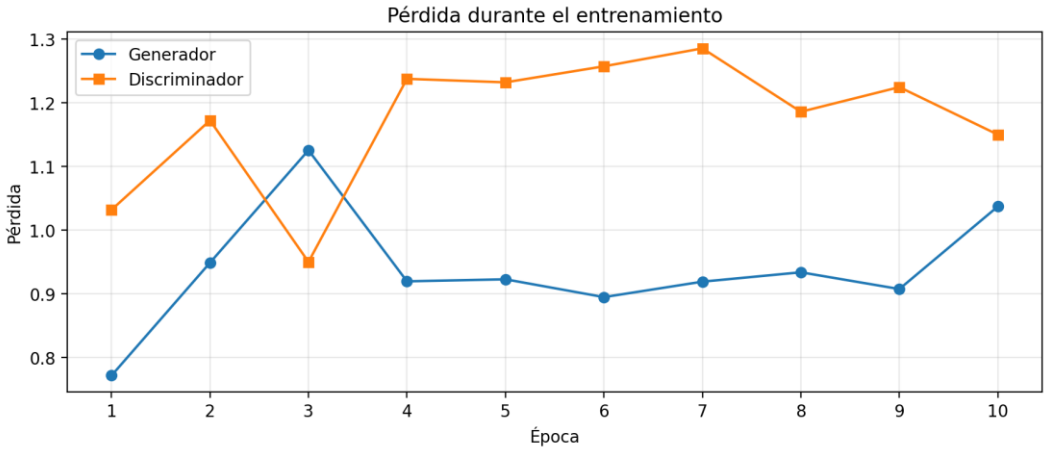
Pérdidas del generador y del discriminador por época

Época	Pérdida G	Pérdida D	Tiempo (s)
1	0.7717	1.0321	19.1
2	0.9490	1.1725	11.9
3	1.1257	0.9502	12.2
4	0.9197	1.2378	12.7
5	0.9229	1.2323	12.7
6	0.8949	1.2575	13.0
7	0.9192	1.2857	13.5
8	0.9339	1.1861	14.1
9	0.9076	1.2248	14.1
10	1.0373	1.1502	13.6

El tiempo total de entrenamiento fue de aproximadamente 135.9 segundos (≈ 2.3 minutos) en GPU T4. La primera época requirió 19.1 s debido a la compilación de la función de entrenamiento con @tf.function; las épocas posteriores se estabilizaron entre 12 y 14 segundos.

Figura 1

Evolución de las pérdidas del generador y del discriminador durante 10 épocas



4.2 Evolución visual de las imágenes generadas

En la época 5, las 16 imágenes producidas por el generador mostraron formas compatibles con dígitos, aunque con trazos discontinuos, ruido de fondo y ausencia de contornos bien definidos. En la época 10 se apreció una mejora cualitativa: los dígitos generados presentaron trazos más continuos y formas más reconocibles (se distinguieron figuras que sugieren 0, 1, 3, 6, 8 y 9), si bien persistieron el desenfoque y ciertos artefactos característicos de un entrenamiento corto.

Figura 2

Imágenes generadas por la GAN al finalizar la época 5

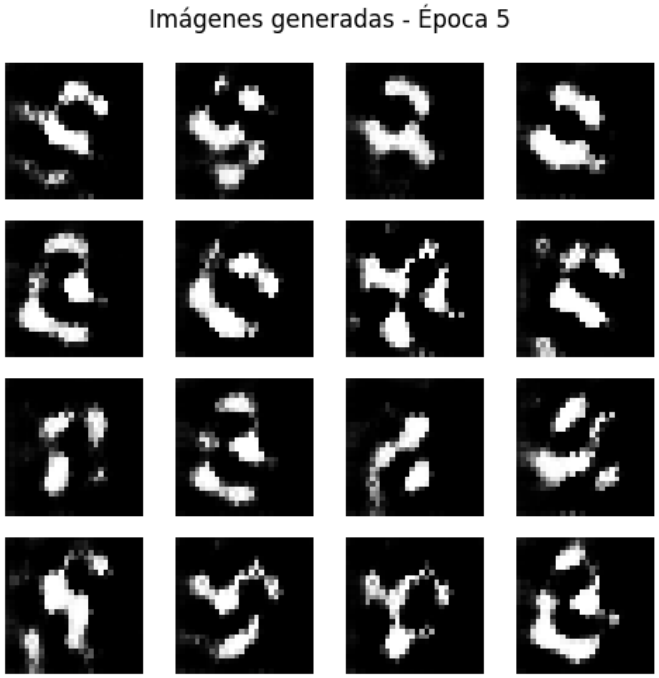
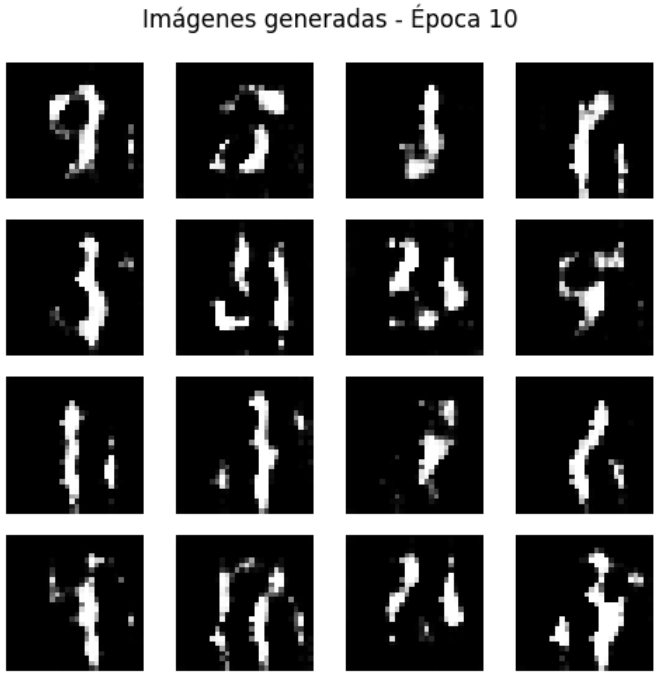


Figura 3

Imágenes generadas por la GAN al finalizar la época 10



5. Análisis de Resultados

5.1 Comportamiento adversarial de las pérdidas

Los valores de la Tabla 3 evidencian la dinámica característica del juego minimax. En la primera época la pérdida del generador fue baja (0.7717) frente a la del discriminador (1.0321), situación esperable porque el discriminador parte de un estado aleatorio. Sin embargo, a partir de la época 3 el discriminador incrementó su capacidad de detección (D = 0.9502), lo que obligó al generador a elevar su pérdida (G = 1.1257). En las épocas 4 a 9 la pérdida del discriminador se mantuvo sistemáticamente por encima de 1.18, mientras que la del generador osciló entre 0.89 y 0.94, reflejando un equilibrio dinámico en el que ambas redes mejoran de manera simultánea sin que ninguna domine por completo.

La ausencia de colapso de modo (mode collapse) se infiere de la variedad de dígitos observada en las muestras de la época 10. Si el generador hubiera colapsado, todas las salidas serían idénticas o muy similares. No obstante, tampoco se observa una convergencia completa: las pérdidas oscilan en lugar de estabilizarse, lo cual es característico de las GAN y explica la calidad limitada de las imágenes.

5.2 Calidad de las imágenes generadas

Las imágenes de la época 10 son reconocibles como dígitos manuscritos, pero no alcanzan el nivel de realismo del dataset original. Este resultado es coherente con el número de épocas empleado (10): la literatura reporta que una DCGAN sobre MNIST requiere entre 50 y 100 épocas para producir dígitos nítidos. La brecha cualitativa observada se explica por tres factores: entrenamiento corto (2 340 actualizaciones por red), capacidad del generador que requiere más iteraciones para explotarse por completo, e inestabilidad intrínseca del entrenamiento adversarial.

5.3 Relación con los conceptos teóricos

Los resultados confirman empíricamente los fundamentos expuestos en la sección de conceptos clave: (i) el generador aprendió a transformar ruido latente en imágenes con estructura de dígitos, evidenciando que efectivamente aproxima la distribución de los datos reales; (ii) el discriminador cumplió su rol de árbitro y su pérdida creciente sugiere que su tarea se volvió progresivamente más difícil, señal de que el generador mejoró; (iii) el equilibrio dinámico de las pérdidas refleja el juego minimax descrito por Goodfellow et al. (2014).

6. Conclusiones

La implementación de una DCGAN sobre el dataset MNIST permitió verificar de forma práctica los principios del aprendizaje adversarial. Las principales conclusiones son:

1. Logro del objetivo: la GAN entrenada durante 10 épocas generó imágenes sintéticas que reproducen la estructura visual de los dígitos manuscritos, con variedad suficiente para descartar el colapso de modo. El objetivo de la práctica se cumplió en un grado inicial, aunque la calidad de las muestras aún no es equiparable a la de los datos reales.

2. Comprensión conceptual: la práctica evidenció la mecánica del juego minimax. Se observó cómo el generador y el discriminador se afectan mutuamente y cómo el equilibrio entre ambos es frágil y dinámico. Asimismo, se comprendió la importancia de hiperparámetros como la tasa de aprendizaje, el tamaño del lote y la arquitectura convolucional para la estabilidad del entrenamiento.

3. Aplicabilidad: los resultados confirman que las GAN son una técnica viable para la generación de datos sintéticos con aplicaciones directas en aumento de datos, síntesis de imágenes médicas, generación de rostros y transferencia de estilo, entre otras.

4. Limitaciones y trabajo futuro: el experimento presenta limitaciones asociadas al número reducido de épocas y al dataset en escala de grises de baja resolución. Como trabajo futuro se propone: (i) extender el entrenamiento a 50–100 épocas; (ii) implementar variantes más estables como WGAN-GP o LSGAN; (iii) aplicar la arquitectura a un dataset en color de mayor resolución (p. ej., CIFAR-10 o CelebA); (iv) incorporar métricas cuantitativas de evaluación como FID (Fréchet Inception Distance) o IS (Inception Score); y (v) comparar el rendimiento con otras arquitecturas generativas como VAE o modelos de difusión.

Repositorio de GitHub: https://github.com/BryanRoa/R2-A5-S11-INFORME-REDES-GAN-DEEP-LEARNING-.git

7. Referencias

Waya.ai. (2017). On the intuition behind deep learning & GANs: Towards a fundamental understanding. Medium. https://medium.com/@waya.ai/introduction-to-gans-a-boxing-match-b-w-neural-nets-b4e5319cc935

DataDrivenInvestor. (2019). *Artificial intelligence + GANs can create fake celebrity faces*. Medium. https://medium.datadriveninvestor.com/artificial-intelligence-gans-can-create-fake-celebrity-faces-44fe80d419f7

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. En Advances in Neural Information Processing Systems (Vol. 27, pp. 2672–2680). Curran Associates. https://arxiv.org/abs/1406.2661

LeCun, Y., Cortes, C., & Burges, C. J. C. (1998). The MNIST database of handwritten digits. AT&T Labs. http://yann.lecun.com/exdb/mnist/

Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv. https://arxiv.org/abs/1511.06434

TensorFlow. (s. f.). Deep convolutional generative adversarial network. TensorFlow Core. Recuperado el 5 de noviembre de 2026, de https://www.tensorflow.org/tutorials/generative/dcgan